

AGENTIC AI

A Production-Ready Guide for Data Scientists & Architects

Chapter 1 · Introduction to Prompting for Effective LLM Reasoning and Planning

Prerequisites & Course Overview

Before diving into the world of Agentic AI, let's ensure you have the right foundation in place. This chapter — and the broader course — is designed with the following knowledge prerequisites in mind:

Required Knowledge — What You Should Already Know

Basic Python Programming

Comfort with Python syntax, data types (strings, lists, dicts), writing functions, and understanding how a Python script flows from top to bottom.

Jupyter Notebook Proficiency

Ability to navigate the Jupyter interface, execute individual code cells, and interpret their outputs. This is essential because all practical exercises use notebooks.

Foundational Understanding of LLMs

You should know what a Large Language Model is at a conceptual level — how it takes text input (a prompt) and generates text output, and its general capabilities like text generation, summarization, and comprehension.

By the end of this course, you will have mastered:

- The architecture and inner workings of AI Agents — how they reason, plan, and interact with digital environments.
- Advanced prompting techniques for precise, nuanced LLM instruction.
- Role-Based Prompting — creating specialized AI personas for targeted outputs.
- Chain-of-Thought (CoT) and ReAct (Reason + Act) prompting to unlock multi-step problem solving.
- Systematic Prompt Instruction Refinement — iteratively improving prompts for optimal performance.
- Building multi-step Agentic Workflows by chaining prompts together to tackle complex, real-world tasks.
- Prompt Chaining & Feedback Loops — robust workflows that allow AI to improve its own outputs iteratively.

SECTION 1

Theory & Conceptual Foundations

1. What Is an AI Agent?

The term 'AI Agent' is used extensively in modern software, but it's important to understand precisely what it means — especially how it differs from simply calling an LLM API. An AI Agent is not just a chatbot. It is an intelligent, autonomous system capable of perceiving its environment, making decisions, and taking goal-directed actions — all without requiring a human to hand-hold each step.

1.1 Core Characteristics of an Agent

Three fundamental characteristics distinguish an AI Agent from a simple LLM call:

The 3 Pillars of Agentic Behavior

1. Autonomy

The agent can operate independently, making decisions and taking sequences of actions toward a goal without explicit per-step human instruction.

2. Adaptability

The agent responds intelligently to new information it receives from its environment. If an action fails, it can reason about why and try a different approach.

3. Interaction with Surroundings

Unlike a passive text generator, an agent can interface with external systems — databases, APIs, web browsers, code executors — to read and write information in the real world.

Traditional AI systems compute or follow static, pre-programmed rules. A conventional LLM, by contrast, is extraordinarily powerful at language generation, but it is fundamentally reactive — it waits passively for your prompt and produces a response. An AI Agent takes this a decisive step further. Rather than just assisting you with a single response, an agent can work alongside you or independently on your behalf, handling tasks that span many sequential steps. The key cognitive leap is that agents can reason, plan, and act toward a goal using tools — they are active participants in completing a task, not passive responders.

1.2 The Five Components of an AI Agent

A modern, production-grade AI Agent is built from five interconnected components. Understanding each of these is essential before you write a single line of code:

Component	Role in the Agent System
LLM (Large Language Model)	The cognitive 'brain' — understands context, reasons about the situation, decides what to do next, and generates language-based plans and responses.

Tools	External functions, APIs, or resources the agent can invoke. Examples: web search, database queries, calculator, send email, run code. Tools give the agent hands.
Instructions (System Prompt)	Explicit behavioral guidelines. The system prompt defines the agent's personality, constraints, goals, available tools, and how it should format its outputs.
Memory	Short-term memory holds the current conversation context. Long-term memory persists information across sessions, letting the agent learn and recall past interactions.
Runtime / Orchestration Layer	The execution engine. It runs the LLM, captures its outputs, executes tool calls on the LLM's behalf, and feeds results back. The LLM itself only generates text — the orchestrator is what actually makes things happen.

Key Insight — Why the Orchestration Layer Matters

A critical misconception is that the LLM 'uses' tools directly. In reality, the LLM outputs text that describes which tool to call and with what parameters. The orchestration layer reads this output, executes the actual tool call (e.g., makes the API request), and then feeds the result back to the LLM as new input. The LLM never directly touches external systems.

1.3 Why Build with Agents? The Limitations of Standalone LLMs

You might reasonably ask: 'If LLMs are so powerful, why do I need agents?' The answer lies in understanding what LLMs fundamentally cannot do on their own:

- LLMs have a fixed knowledge cutoff date — they cannot access real-time or proprietary data.
- LLMs cannot take actions in the world — they cannot send emails, query live databases, or execute code.
- LLMs struggle with very long, multi-step tasks in a single prompt — context gets confused, details are missed.
- LLMs cannot verify their own outputs — they have no mechanism to check if what they said was actually true or correct.

Agents bridge every one of these gaps. By wrapping an LLM with tools, planning mechanisms, memory, and an orchestration layer, you transform a passive text generator into an active problem-solver that can navigate complex, multi-step workflows and interact meaningfully with the real world.

2. The Critical Role of Prompting

The LLM is the brain of your agentic system. But how do you instruct that brain to perform complex reasoning, make intelligent decisions, and interact correctly with tools and its environment? This is where effective prompting becomes not just important — but the primary engineering lever you have.

2.1 What Is a Prompt?

A prompt is a set of instructions provided to an LLM that customizes, enhances, or refines its behavior and capabilities. For simple use cases, a prompt is just a question. For agents, prompts are the primary mechanism by which we 'program' the LLM — they define everything from the agent's persona and goals to the exact format it should use when deciding to call a tool.

The Programming Analogy

Think of the LLM as a powerful but instruction-less computer processor. The prompt is the software — the code — that tells the processor what to actually do. Without precise instructions, the processor (LLM) will default to generic, often unhelpful behavior. The quality of your prompt directly determines the quality of your agent's behavior.

For agentic systems, prompts include three distinct layers:

- **System Prompts** — Define the agent's overall role, behavior, constraints, and the tools available to it.
- **User Requests** — The specific task or question posed in a given turn.
- **Tool Descriptions** — Instructions embedded in the prompt that describe what each tool does, when to use it, and what parameters it requires.

3. Real-World Use Case: Processing a Refund

Let's ground these abstract concepts in a concrete example. Imagine a customer support AI agent tasked with processing a refund request. When a customer submits the message 'I'd like a refund for Order #12345 — the product arrived damaged,' this becomes the initial prompt or input to the agentic system.

Instead of following a rigid, pre-coded script, the agent uses its LLM reasoning capabilities to dynamically break this complex request into smaller, manageable sub-tasks — a process called planning. Here is how a well-designed agent would reason through this:

Agent's Step-by-Step Reasoning for a Refund Request

Step 1 — Identify Information Needed

Determine that to process a refund, the agent needs the customer's purchase history and order details.

Step 2 — Use a Tool to Fetch Data

Call the `get_order_details(order_id='12345')` tool to retrieve order information from the database.

Step 3 — Apply Policy Logic

Check the product details against the company's refund policy (e.g., is it within the return window? Is 'damaged on arrival' a valid reason?).

Step 4 — Make a Decision

Based on the findings — e.g., order was placed 5 days ago and policy allows 30-day returns for damaged goods — decide to approve the refund.

Step 5 — Execute and Respond

Call the `process_refund()` tool, then respond to the customer with a confirmation message.

This entire process — perceiving the input, planning a sequence of steps, calling tools, observing results, and producing a final output — is what distinguishes an AI Agent from a simple chatbot. Crucially, all of this behavior is shaped and controlled through prompting.

4. Role-Based Prompting (Agent Personas)

Large Language Models are trained on enormous, diverse datasets and have broad, general knowledge. This breadth is a strength — but it also means that without guidance, the LLM may respond in any one of dozens of possible styles: formal, casual, technical, humorous, literary. For production AI systems, this unpredictability is a problem. Role-Based Prompting solves it.

4.1 The Core Idea

Role-Based Prompting means giving your LLM a specific identity — a persona — to inhabit during the interaction. Think of it like casting an actor in a play. You don't just hand them lines; you give them character background, motivation, and direction on how to deliver those lines. The LLM's 'character background' and 'direction' are provided in the prompt — typically in the system message.

The goal is to produce outputs that are specialized, consistent, and aligned with the defined role. Consider the simple task 'Tell me about cats.' Without a role, the LLM might respond generically:

`Generic: "Cats are common pets known for being playful and independent."`

But with a role defined, you get targeted, appropriate outputs:

`Literary Critic: "As celebrated companions, cats are lauded for their playful antics and self-reliant nature."`

`Pirate: "Ar, cats be beloved mates, prized fer their cheeky antics and self-reliant spirit, yar!"`

4.2 Why Is Role-Based Prompting So Effective?

Without a defined role, the LLM faces an enormous solution space when generating a response — any tone, any style, any level of technicality could be 'correct.' By assigning a role, you dramatically narrow this solution space. The role acts as a strong prior that biases the model toward outputs

consistent with the defined persona's expertise, communication style, and vocabulary. The result is more targeted, more consistent, and often more useful output.

4.3 Anatomy of a Role-Based Prompt

A well-crafted role-based prompt is built from up to five components. Note that not every prompt needs all five — it depends on the complexity of the task:

Component	Description & Purpose
[Role]	The persona the LLM adopts. Sets tone, style, expertise level, and communication approach. Example: 'You are a Senior Python Developer specializing in efficient algorithms.'
[Task]	The specific action to perform. Should be precise and unambiguous. Example: 'Write a function called fibonacci_memoized that takes integer n and returns the n-th Fibonacci number.'
[Output Format]	How the response should be structured. Example: 'Respond in JSON format with keys dish_title and description.' Or: 'Output a single sentence in Markdown.'
[Examples]	Few-shot examples showing the desired input-output pattern. These dramatically improve consistency. Example: 'Q: 1+3? / A: Tis 4, yar!'
[Context]	Background information the LLM needs that is not in its training data. Example: current date, user's previous orders, company-specific policies.

Example: A Complete Role-Based Prompt

Act like a pirate! #[Role]

Perform the calculation: 1 + 1? #[Task]

Output a sentence in Markdown. #[Output Format]

Here's an example: Q: 1 + 3? / A: "Tis 4, yar!" #[Example]

Expected Output: "Ahoy! That be simple reckonin', matey! Tis 2, yar!"

Note: The tone and calculation are correct but it used two sentences. Iteratively refining the prompt (e.g., adding 'USE ONLY ONE SENTENCE') will fix this.

5. Evaluating Agent Personas

Defining a persona is only half the battle. You must evaluate whether your agent is actually staying in character, producing accurate outputs, and meeting your quality criteria. Evaluation in the context of LLM-based systems is analogous to a director assessing an actor's performance during rehearsals — you need a structured process for measuring whether the AI's outputs meet expectations.

5.1 Ground Truth Evaluation

The most reliable evaluation method is creating a ground truth dataset — a set of test inputs paired with ideal, manually curated outputs. The process works as follows:

1. Define success criteria: What does 'good' output look like? Does it require factual accuracy, a specific tone, a particular JSON schema, a certain length?
2. Build a test set: Create representative input examples and manually write the ideal outputs for each.
3. Generate agent outputs: Run your agent against the test inputs.
4. Compare outputs to ground truth: Use automated checks for structured data (JSON schema validation, field presence) or LLM-as-a-Judge for freeform text.
5. Analyze traces: Always inspect the raw input-output sequences (called traces) — automated metrics can miss subtle errors that become obvious when you read the conversation.

5.2 LLM-as-a-Judge

For freeform text outputs (where exact string matching is impossible), a powerful technique is using a second LLM as the evaluator. You feed the evaluator LLM both the agent's output and the ground truth, along with your scoring rubric (e.g., 'Rate this response 1-5 for tone accuracy, factual correctness, and format adherence'). The evaluator returns a structured score. Important caveat: you must then evaluate the evaluator — test it against a smaller set of human-labeled examples to ensure its scores are calibrated.

5.3 Other Evaluation Dimensions

Additional Evaluation Methods Beyond Ground Truth

Consistency & Persona Adherence

Does the agent stay in its defined role even when asked off-topic questions? E.g., a hospital assistant asked about cooking should decline gracefully, not answer as a chef.

Robustness Testing

How does the agent respond to adversarial inputs, prompt injection attempts, or unexpected edge cases? Critical for agents handling sensitive data.

Simple Operational Metrics

Track median response length, latency, token usage, and cost. These metrics help you optimize usability and control operational expenses in production.

Evaluation should not be a one-time activity. Track performance continuously — both during development and in production. Real-world data will surface failure modes that your test set missed. Treat LLM application development like software engineering: test early, test often, and iterate based on evidence.

6. Chain-of-Thought (CoT) and ReAct Prompting

Even with a well-defined role, LLMs can struggle with complex, multi-step problems. Simply asking 'What is the fastest route across Tokyo right now?' will fail — the LLM has no access to live traffic data and no mechanism to query a maps API. This section introduces two prompting strategies that dramatically expand what LLMs can tackle: Chain-of-Thought and ReAct.

6.1 The Problem: LLMs as Isolated Scholars

Imagine asking a brilliant but completely isolated scholar — someone with vast knowledge of everything written up until a certain date, but with no phone, no internet, and no ability to interact with the outside world — to answer a question requiring real-time data. They can answer historical questions perfectly, but they fail utterly on questions that require current information or multi-step interaction with external systems. This is precisely the situation with a standalone LLM.

6.2 Chain-of-Thought (CoT) Prompting

Chain-of-Thought prompting is a technique that encourages the LLM to generate a sequence of explicit intermediate reasoning steps before producing its final answer. Rather than jumping directly from question to answer (which often leads to errors on complex tasks), CoT guides the model to 'think out loud' — making its reasoning process transparent and step-by-step.

Types of Chain-of-Thought

- **Zero-Shot CoT:** The simplest form. Simply append 'Let's think step by step' to your prompt. Remarkably, this single instruction dramatically improves performance on reasoning tasks.
- **Few-Shot CoT:** Provide example problems with their step-by-step reasoning and final answer. The LLM learns the reasoning pattern from your examples and applies it to new problems.

Zero-Shot CoT Example

Without CoT: 'Is $3 + 1 = 4$ correct?' → 'Yes.' (could be right by chance)

With CoT: 'Is $3 + 1 = 4$ correct? Work out your own solution step by step, then compare it to mine.'
→ 'Let me compute: $3 + 1 = 4$. The student's answer of 4 is correct.'

The step-by-step instruction prevents the model from lazily echoing the student's answer without verification.

Benefits of CoT

- **Improved Accuracy:** By breaking problems into steps, the model makes fewer arithmetic, logical, and commonsense reasoning errors.
- **Reduced Hallucination:** Explicit reasoning forces the model to commit to intermediate conclusions, making it harder to fabricate a plausible-sounding but wrong answer.
- **Interpretability:** You can read the model's reasoning chain to understand exactly where it went right or wrong — invaluable for debugging.
- **Transparency:** Moves the LLM from a 'black box' to an inspectable reasoning system that builds trust with stakeholders.

6.3 ReAct (Reason + Act) Prompting

CoT is powerful for improving internal reasoning, but it still cannot help the LLM access new information or interact with the world. ReAct (Reason + Act) extends CoT by interleaving reasoning steps with action steps — allowing the LLM to plan tool use, execute those tools (via the orchestrator), and incorporate the results into subsequent reasoning.

The ReAct loop has three phases that repeat iteratively until the task is complete:

The ReAct Loop: Thought → Action → Observation

THOUGHT

The model internally reasons about the current state of the task. It analyzes what it knows, what it still needs to find out, and decides on the next specific action to take. This is pure text — it never leaves the LLM.

ACTION

Based on the thought, the model specifies a tool call in a structured format (e.g., JSON) that the orchestration layer can parse and execute. Tools are external functions the agent can invoke.

OBSERVATION

The orchestrator runs the tool and returns the result to the LLM. This new information — e.g., weather data, search results, a database record — is appended to the conversation history and feeds directly into the next THOUGHT.

Each iteration of this loop is recorded in the conversation's message history. The LLM bases every new THOUGHT on the complete accumulated history of all previous thoughts, actions, and observations. This gives the agent a working memory of everything it has done and learned during the task. The loop terminates when the agent calls a special `final_answer` tool, signaling that it has gathered all necessary information and is ready to respond.

6.4 Designing ReAct Prompts

A ReAct system prompt must accomplish several things simultaneously: define the agent's role, describe the available tools with precise input/output specifications, and teach the agent the THOUGHT/ACTION format through examples. These prompts are typically long and detailed — do not underestimate the importance of thorough tool descriptions. Here is a real-world example of a ReAct system prompt structure for a financial analyst agent:

ReAct System Prompt Structure — Financial Analyst Agent

ROLE:

You are a diligent financial analyst assistant. You help users gather and compare stock information and news.

REASONING FORMAT (Required):

At each step, respond with exactly one THINK/ACT block:

THINK: Reason about what you know and what the next logical step is.

ACT: Call exactly one tool from the list below.

AVAILABLE TOOLS:

get_stock_quote(ticker: str) — Gets the latest stock info for a ticker symbol.

search_financial_news(company_name: str) — Searches recent news for a company.

final_answer(summary: str) — Use ONLY when you have all needed information.

EXAMPLE INTERACTION:

User: What is MegaCorp's (\$MC) P/E ratio?

THINK: I need the stock data for \$MC to find the P/E ratio.

ACT: get_stock_quote(ticker='\$MC')

[OBSERVATION: P/E ratio is 25.4]

THINK: I have the P/E ratio. I can now return the answer.

ACT: final_answer(summary='MegaCorp's P/E ratio is 25.4')

7. Prompt Instruction Refinement

Prompt engineering is not a one-shot activity. It is an iterative, scientific process of writing a prompt, observing the output, identifying what went wrong, adjusting specific components, and repeating — much like tuning a musical instrument. A small change in wording can dramatically shift the LLM's response. Understanding which components to modify and in what direction is a core skill for anyone building production AI systems.

7.1 The Tunable Components

Every prompt has distinct components that each control different aspects of the output. The key insight is to modify one component at a time — this is the systematic, scientific approach to prompt engineering:

Component	What Changing It Does
Role	Changes the persona, tone, style, and expertise level of the response. 'Helpful assistant' vs. 'Skeptical historian' produces dramatically different outputs for the same task.
Task	Alters the core action and its constraints. 'Summarize this text' vs. 'Identify the three main arguments' vs. 'Write a tweet about this' are very different instructions.
Output Format	Dictates the structure of the response. 'Respond in JSON' vs. 'Use bullet points' vs. 'Write a single paragraph' controls how information is organized.
Examples (Few-Shot)	Refines style, detail level, vocabulary, and quality by showing the model the exact pattern you want. Often the single most impactful component.
Context	Modifies the background information available to the model. Different context leads to different answers — e.g., providing last month's sales data vs. last year's.

Important: Components Are Not Independent

Changing the Output Format may also affect the Task behavior (e.g., asking for JSON output may make the model more terse in its descriptions). Changing the Role may alter the vocabulary used even when the Task is identical. Keep this in mind when debugging unexpected outputs — the interaction between components may be the cause.

7.2 Prompt Refinement Best Practices

6. Write Clear and Detailed Instructions — Be precise and descriptive. Do not assume the model infers what you meant. State exactly what you want.
7. Use Roles or Personas — Assign a specific role to establish tone, expertise, and communication style upfront.
8. Break Down Complex Tasks — Divide multi-step problems into smaller, sequential sub-tasks. Ask the model to 'think step by step' for reasoning-intensive parts.
9. Specify Output Format and Constraints Explicitly — Define structure (JSON, bullet points, numbered list), length limits (under 75 words), and language constraints.
10. Include Few-Shot Examples — When the desired output pattern is non-obvious, providing 2-3 concrete input-output examples dramatically improves consistency.

11. **Provide All Relevant Context** — Give the model information it cannot know from training data alone — e.g., your company's return policy, today's date, the user's account status.
12. **Optimize Tool Descriptions (for Agents)** — Each tool description in the system prompt must be clear about what the tool does, when to use it, and what its parameters mean. The LLM decides which tool to call based solely on these descriptions.

7.3 Common Pitfalls to Avoid

Prompting Anti-Patterns — What Not to Do

Ambiguity — Unclear instructions produce unpredictable outputs. Future model versions also won't 'know' what you implicitly meant.

Insufficient Context — Expecting the LLM to know things you didn't tell it leads to hallucinations and errors.

Too Much Competing Context — Overwhelming the prompt with irrelevant information or contradictory instructions degrades performance. Focus matters.

Poor Tool Descriptions — If tool descriptions are vague, the LLM won't know when or how to use them correctly.

Expecting Magic — LLMs are probabilistic systems, not omniscient. Precision in your instructions is not optional.

Ignoring Adversarial Risks — Prompts can be exploited for injection attacks, jailbreaks, and data leakage. Design your prompts defensively.

8. Prompt Chaining & Agentic Workflows

Individual prompts, however well-crafted, have limitations when dealing with genuinely complex, multi-stage tasks. A single LLM call trying to research a topic, summarize the findings, and draft a LinkedIn post — all at once — will produce confused, low-quality results. The solution is to break the task into independent sub-tasks and chain them together programmatically. This is the foundation of agentic workflows.

8.1 Sequential Prompting

Sequential Prompting means decomposing a complex goal into a series of smaller, focused sub-tasks, each handled by its own dedicated prompt. This mirrors how humans tackle complex problems — one step at a time, with full attention on each step. Instead of:

```
'Research AI agents, summarize the key concepts, and write a LinkedIn post about them.'
```

You break it into three sequential prompts:

13. Prompt 1: 'Research and list the key concepts of AI agents.' → RESPONSE_1
14. Prompt 2: 'Summarize the following key concepts in 3 bullet points: {RESPONSE_1}' → RESPONSE_2

15. Prompt 3: 'Draft a professional LinkedIn post based on this summary: {RESPONSE_2}' → FINAL_RESPONSE

Each step can be independently tested and refined. If the final LinkedIn post is poor, you can debug which step introduced the problem — was the research weak? Was the summary too terse? This modularity is enormously valuable in production systems.

8.2 Prompt Chaining with Gate Checks

Prompt chaining alone is not sufficient for production reliability. LLMs can hallucinate, produce incorrect JSON, fail to follow format instructions, or make factual errors — and an error in step 1 cascades through every subsequent step (the 'domino effect'). Gate Checks are programmatic validation steps placed between each link in the chain.

How Gate Checks Work

After each LLM call, before passing its output to the next step:

✓ If the check PASSES → continue to the next step.

✗ If the check FAILS → halt and raise an error, OR automatically retry (optionally including the failure reason in the retry prompt), OR loop for a maximum N iterations.

Types of Gate Checks

- **Format Checks:** Is the output valid JSON? Does it contain all required fields? Is it within the specified length? Libraries like Pydantic are excellent for this.
- **Content Checks:** Are required keywords present? Is the text on-topic? Does the generated response stay within scope? Regex or semantic embeddings can help.
- **Logic Checks:** Are numerical values reasonable? Does generated code compile without syntax errors? (Use Python's ast module or a linter for code validation.)
- **Safety Checks:** Does the output contain harmful, biased, or off-policy content? Another LLM can serve as a safety evaluator.

9. LLM Feedback Loops

Even with gate checks, a single pass through a prompt chain may not produce a perfect result. Feedback loops take prompt chaining to the next level by allowing the LLM to receive structured feedback, reflect on that feedback, and iteratively revise its output — all within a single automated workflow, without any human intervention.

9.1 The Iterative Advantage

Think about how you write a professional email. Your first draft is often too blunt or informal. You read it, identify the problem, and revise. Then you re-read. Perhaps you revise again. Each revision is driven by feedback from your own evaluation. Feedback loops automate exactly this process for LLM-generated content.

The fundamental mechanism is still prompt chaining — but now, the chain includes a loop. The output of step N feeds into an evaluation step, which generates feedback, which is incorporated into a revised prompt for another iteration of step N. This repeats until the output passes all checks or a maximum iteration count is reached.

9.2 Sources of Feedback

Four Ways an Agent Can Receive Feedback

1. Self-Correction

Prompt the same LLM to evaluate its own previous output. E.g., 'Review the following email for tone and politeness. List specific issues.' Then feed its critique back as the prompt for a revision.

2. External Tool Results

If the agent generates code, actually execute it. If it throws an exception, feed the error message back as feedback. This is one of the most powerful feedback mechanisms — objective, immediate, and unambiguous.

3. Programmatic Validation

Gate checks that fail provide structured feedback — e.g., 'Output is missing required field email_address.' This message goes back into the next iteration's prompt.

4. Human-in-the-Loop

In human-supervised workflows, a user can review an intermediate output and provide natural language feedback. The agent incorporates this and continues.

9.3 Monitoring Feedback Loops

Building a feedback loop is only valuable if you can tell whether it's actually improving outputs. Always instrument your loops with logging:

- Log every iteration: the prompt sent, the LLM's response, the evaluation result, the feedback generated.
- Track quality metrics across iterations: are outputs improving? At what iteration does quality plateau?
- Set a maximum iteration limit: prevent infinite loops by capping at N iterations (typically 3-5 is sufficient for most tasks).
- Alert on degradation: if iteration 3 is worse than iteration 1, something is wrong with your feedback prompt.

SECTION 2

Practical Implementation & Coding Walkthroughs

This section walks through the practical implementation of every concept covered in Section 1. Each notebook is broken down cell-by-cell with comprehensive explanations of what the code does, why it is written that way, and what to look for in the output.

Notebook 1: Implementing Role-Based Prompting with Python

This notebook demonstrates how to use the OpenAI Python SDK to send structured role-based prompts to an LLM and observe how changing the role component affects output style and content.

In [1]: — Cell 1

```
# Install required packages (run once)
!pip install openai
```

 *Explanation: This cell installs the OpenAI Python library, which provides a high-level SDK for interacting with OpenAI's API. The exclamation mark prefix is Jupyter's syntax for running shell commands. You only need to run this once per environment. If you're using a virtual environment or a cloud notebook (Google Colab, Kaggle), this may already be installed.*

In [2]: — Cell 2

```
import openai
import os

# Best practice: Load your API key from an environment variable, never hardcode it
client = openai.OpenAI(api_key=os.environ.get('OPENAI_API_KEY'))
```

 *Explanation: Here we import the openai library and the os module. The os.environ.get('OPENAI_API_KEY') call reads your API key from an environment variable rather than hardcoding it in the script. This is a critical security best practice — hardcoding API keys in code is a common source of credential leaks, especially when pushing code to GitHub. Set your environment variable beforehand with: export OPENAI_API_KEY='your-key-here' (Mac/Linux) or set OPENAI_API_KEY=your-key-here (Windows). We instantiate an OpenAI client object that we will reuse for all API calls.*

In [3]: — Cell 3

```
def call_llm(system_prompt: str, user_prompt: str, model: str = 'gpt-4o') -> str:
    """
    Sends a system + user prompt to the LLM and returns the text response.

    Args:
        system_prompt: The role/instructions (defines the agent's behavior)
        user_prompt:   The specific task or question
        model:          The OpenAI model to use (default: gpt-4o)
    Returns:
        The model's response as a plain string
    """
    response = client.chat.completions.create(
```

```

    model=model,
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt}
    ]
)
return response.choices[0].message.content

```

 **Explanation:** This is our core utility function — a reusable wrapper around the OpenAI Chat Completions API. The messages list is the critical component: it contains two message objects. The 'system' message is where we place our role definition and behavioral instructions — the LLM treats this with the highest priority. The 'user' message contains the actual task or question. The response object is a complex nested structure; response.choices[0].message.content extracts just the text of the first (and typically only) generated response. By wrapping this in a function, we avoid repeating this boilerplate in every cell and make our code cleaner and more maintainable.

In [4]: — Cell 4

```

# — Baseline: No role defined —————
system_no_role = "You are a helpful assistant."
user_task = "Tell me about cats."

output_baseline = call_llm(system_no_role, user_task)
print('=== BASELINE (No Specific Role) ===')
print(output_baseline)

```

 **Explanation:** This cell establishes our baseline — what does the LLM produce with a completely generic system prompt ('You are a helpful assistant') and a simple, open-ended task? The output will be informative but generic, with no particular personality or expertise angle. This gives us a reference point for comparison. When you run this, notice the neutral, factual tone and the lack of any stylistic differentiation. This is what happens when you don't give the LLM a role to inhabit.

In [5]: — Cell 5

```

# — Role 1: Literary Critic —————
system_literary = """You are a celebrated literary critic and essayist with a
refined, eloquent writing style. You bring a poetic sensibility to even mundane
topics, drawing connections to literature and culture."""

output_literary = call_llm(system_literary, user_task)
print('=== ROLE: Literary Critic ===')
print(output_literary)

# — Role 2: Marine Biologist —————
system_scientist = """You are a Marine Biologist specializing in feline
evolutionary biology. You explain topics with scientific precision, citing
evolutionary context and biological mechanisms. Use technical but accessible
language. Include specific scientific facts."""

output_scientist = call_llm(system_scientist, user_task)
print('\n=== ROLE: Marine Biologist ===')
print(output_scientist)

```

 **Explanation:** Now we introduce two distinct roles and run the same 'Tell me about cats' task against each. The Literary Critic role should produce eloquent, stylized prose with cultural references. The Marine

Biologist role should produce precise, scientific language with evolutionary context. By running the exact same user_task through both roles, we create a controlled experiment — the only variable changing is the system prompt (the role). Compare the vocabulary, sentence structure, depth of technical detail, and tone between the two outputs. This is the most direct demonstration of role-based prompting's power.

In [6]: — Cell 6

```
# — Full structured role-based prompt: Senior Python Developer —

system_dev = """You are a Senior Python Developer specializing in efficient
algorithms and clean code practices. You write robust, well-documented Python
code. Always include:
- Type hints on all function signatures
- A comprehensive docstring (Args, Returns, Raises, Example)
- PEP 8 compliant formatting
- Comments explaining non-obvious logic"""

user_fibonacci = """Write a Python function called fibonacci_memoized that:
- Takes an integer n as input
- Returns the n-th Fibonacci number
- Uses memoization for efficiency
- Handles edge cases: n=0 returns 0, n=1 returns 1
- Raises ValueError for negative inputs"""

output_fibonacci = call_llm(system_dev, user_fibonacci)
print(output_fibonacci)
```

 *Explanation: This cell applies role-based prompting to a realistic software engineering task. The system prompt is now quite detailed — it doesn't just say 'you are a developer,' it specifies the exact standards the developer adheres to (type hints, docstrings, PEP 8, comments). The user prompt is equally specific — it breaks down the exact requirements of the function including edge case behavior and error handling. Notice how the specificity of both prompts combines to produce production-quality code. If you ran the same user_fibonacci prompt with a generic system prompt, you'd get functional code but likely without type hints, a proper docstring, or edge case handling. The role shapes the quality bar, not just the style.*

In [7]: — Cell 7

```
# — Evaluation: Check the output meets our criteria —
def evaluate_code_output(code_output: str) -> dict:
    checks = {
        'has_type_hints': '-> int' in code_output or ': int' in code_output,
        'has_docstring': '"""' in code_output,
        'has_memoization': 'cache' in code_output.lower() or
                           'lru_cache' in code_output or
                           'memo' in code_output.lower(),
        'handles_edge_0': 'n == 0' in code_output or 'n <= 0' in code_output,
        'raises_valueerror': 'ValueError' in code_output,
    }
    score = sum(checks.values())
    print(f'Quality Score: {score}/{len(checks)}')
    for criterion, passed in checks.items():
        status = '✓ PASS' if passed else 'X FAIL'
        print(f' {status} {criterion}')
    return checks
```

```
evaluate_code_output(output_fibonacci)
```

 *Explanation: This cell implements a simple but practical programmatic evaluator for our LLM's code output. Rather than visually reading the output to judge quality, we write a function that checks for specific, objective criteria: presence of type hints, docstrings, memoization logic, edge case handling, and error raising. Each criterion returns True or False, giving us a quality score out of 5. This is a simplified version of the 'ground truth evaluation' discussed in the theory section — we're not comparing to a ground truth output, but we are applying objective criteria programmatically. In production systems, you'd extend this to use Python's ast module for proper syntax validation or run the function against a test suite.*

Notebook 2: Implementing CoT and ReAct Prompting with Python

This notebook demonstrates Chain-of-Thought prompting and then builds a full ReAct agent loop from scratch — without any agent frameworks — so you understand exactly what is happening under the hood.

 In [1]: — Cell 1

```
# — Zero-Shot Chain-of-Thought Example —
problem = 'A store sells apples for $1.50 each and oranges for $2.00 each.'
problem += ' You buy 4 apples and 3 oranges. How much change do you get from $20?'

# Without CoT
output_no_cot = call_llm(
    'You are a helpful math assistant.',
    problem
)
print('=== WITHOUT CoT ===')
print(output_no_cot)

# With Zero-Shot CoT — simply append 'think step by step'
output_with_cot = call_llm(
    'You are a helpful math assistant.',
    problem + ' Think step by step before giving your final answer.'
)
print('\n=== WITH Zero-Shot CoT ===')
print(output_with_cot)
```

 *Explanation: This cell runs a multi-step arithmetic word problem with and without Chain-of-Thought prompting. The problem has several sequential calculations (cost of apples, cost of oranges, total, change from \$20) that are easy to get wrong if the model tries to answer in one shot. Without CoT, the model may produce a direct answer that skips intermediate steps and risks arithmetic errors. With the single addition of 'Think step by step before giving your final answer,' the model will explicitly compute each sub-calculation before arriving at its answer. Compare both outputs — the CoT version's reasoning chain shows every intermediate computation, making it easy to verify correctness and catch any mistakes. This single-phrase addition costs almost nothing but dramatically improves reliability on multi-step reasoning tasks.*

In [2]: — Cell 2

```
import json
import re

# — Define our 'tools' as plain Python functions —
def get_weather(city: str) -> dict:
    """Simulates a weather API call. In production, replace with a real API."""
    mock_data = {
        'london': {'temp': 15, 'condition': 'Cloudy', 'humidity': 75},
        'new york': {'temp': 22, 'condition': 'Sunny', 'humidity': 55},
        'tokyo': {'temp': 28, 'condition': 'Humid', 'humidity': 80},
    }
    return mock_data.get(city.lower(), {'error': f'No data for {city}'})

def calculate(expression: str) -> dict:
    """Safely evaluates a mathematical expression string."""
    try:
        result = eval(expression, {'__builtins__': {}}, {})
        return {'result': result}
    except Exception as e:
        return {'error': str(e)}

# Registry maps tool names to callable functions
TOOL_REGISTRY = {
    'get_weather': get_weather,
    'calculate': calculate,
}
```

 **Explanation:** This cell defines the 'tools' our ReAct agent will have access to. In a real production system, these would be actual API calls, database queries, or service integrations. Here, `get_weather` returns mock weather data and `calculate` evaluates a math expression. The `TOOL_REGISTRY` dictionary is the bridge between the LLM's text output (which will name a tool by string) and the actual Python function to execute. This is how the orchestration layer works: the LLM says 'call `get_weather` with `city=london`', the orchestrator looks up '`get_weather`' in `TOOL_REGISTRY`, and calls the actual Python function with the provided arguments. Notice that `calculate` uses a restricted `eval` — the `__builtins__: {}` prevents code injection attacks, making it safer than a bare `eval()`.

In [3]: — Cell 3

```
# — ReAct System Prompt —
REACT_SYSTEM_PROMPT = """
You are a helpful assistant with access to the following tools:

1. get_weather(city: str)
   - Returns current weather data for a city
   - Example: get_weather(city='London')

2. calculate(expression: str)
   - Evaluates a mathematical expression
   - Example: calculate(expression='15 * 1.8 + 32')

3. final_answer(answer: str)
```

- Use this when you have all needed information to respond to the user
- Example: `final_answer(answer='The weather in London is 15°C and cloudy.')`

At EVERY step you MUST respond with EXACTLY this format:

THOUGHT: [Your reasoning about what to do next]

ACTION: [tool_name(param='value')]

Do NOT include anything outside this THOUGHT/ACTION format.

"""

 *Explanation: This is the heart of the ReAct implementation — the system prompt that teaches the LLM the THOUGHT/ACTION protocol. It has three critical components: (1) Tool descriptions — precise enough that the LLM knows exactly when to use each tool and what arguments to pass. (2) Format instructions — strict rules requiring every response to follow the THOUGHT:/ACTION: pattern. This is what allows our orchestrator to reliably parse the LLM's output and execute the right tool. (3) The final_answer tool — this is how the agent signals loop termination. Without this, the agent would loop indefinitely. Notice how explicit the format instruction is: 'Do NOT include anything outside this THOUGHT/ACTION format.' LLMs need very explicit formatting constraints to behave consistently, especially for machine-parseable output.*

In [4]: — Cell 4

```
def run_react_agent(user_query: str, max_iterations: int = 5) -> str:
    """
    Runs the full ReAct loop for a given user query.
    Returns the final answer string.
    """
    # message_history maintains the full conversation, including all
    # thoughts, actions, and observations — the agent's 'working memory'
    message_history = [
        {'role': 'system', 'content': REACT_SYSTEM_PROMPT},
        {'role': 'user', 'content': user_query}
    ]

    for iteration in range(max_iterations):
        print(f'\n--- Iteration {iteration + 1} ---')

        # Step 1: Ask the LLM for its next THOUGHT and ACTION
        response = client.chat.completions.create(
            model='gpt-4o',
            messages=message_history
        )
        assistant_message = response.choices[0].message.content
        print(assistant_message)

        # Step 2: Parse the ACTION from the LLM's response
        action_match = re.search(r'ACTION:\s*(\w+)\s*(((.*)\s*)\s*)', assistant_message)
        if not action_match:
            print('ERROR: Could not parse ACTION from response.')
            break

        tool_name = action_match.group(1).strip()
        tool_args_str = action_match.group(2).strip()
```

```

# Step 3: Check for loop termination (final_answer)
if tool_name == 'final_answer':
    print('\n=== AGENT COMPLETE ===')
    return tool_args_str # Return the final answer

# Step 4: Execute the tool if it exists in our registry
if tool_name not in TOOL_REGISTRY:
    observation = f'ERROR: Unknown tool: {tool_name}'
else:
    # Parse keyword arguments: name='value' pattern
    kwargs = {}
    for m in re.finditer(r"(\w+)=['\"]([^\"]+)['\"]", tool_args_str):
        kwargs[m.group(1)] = m.group(2)
    tool_result = TOOL_REGISTRY[tool_name](**kwargs)
    observation = json.dumps(tool_result)

# Step 5: Append the assistant's message AND the observation
# to message_history so the LLM can see it in the next iteration
message_history.append({'role': 'assistant', 'content': assistant_message})
message_history.append({'role': 'user', 'content': f'OBSERVATION: {observation}'})

return 'Max iterations reached without a final answer.'
```

 *Explanation: This is the full ReAct orchestration loop — the most important cell in this notebook. Let's walk through it step by step. (1) message_history is initialized with the system prompt and user query. This list grows with each iteration, giving the LLM complete memory of everything that has happened. (2) In each iteration, we call the LLM with the full accumulated message history. The LLM responds with a THOUGHT/ACTION pair. (3) We use a regular expression to parse the tool name and arguments from the ACTION line. This is why the system prompt enforces a strict format — unstructured output would make this parsing fragile or impossible. (4) If the tool is final_answer, we terminate the loop and return the answer. This is the exit condition. (5) Otherwise, we look up the tool in TOOL_REGISTRY, parse its keyword arguments, and call it. The result becomes the OBSERVATION. (6) Both the assistant's message AND the observation are appended to message_history. This is crucial — the LLM must see both what it said AND what the tool returned to reason correctly in the next iteration. The max_iterations parameter prevents infinite loops.*

In [5]: — Cell 5

```

# — Test the ReAct Agent —

query = """What is the current weather in Tokyo?
Also, if the temperature in Celsius is T,
what is it in Fahrenheit? (Formula: F = T * 1.8 + 32)"""

final_answer = run_react_agent(query)
print(f'\nFINAL ANSWER: {final_answer}')
```

 *Explanation: This cell tests the complete agent with a two-part query that requires both a weather lookup AND a calculation. Watch the output carefully — you should see the agent's full reasoning process play out across 3 iterations: Iteration 1: THOUGHT identifies needing weather data → ACTION calls get_weather(city='Tokyo') → OBSERVATION returns the temperature. Iteration 2: THOUGHT identifies needing to convert Celsius to Fahrenheit → ACTION calls calculate(expression='28 * 1.8 + 32') → OBSERVATION returns 82.4. Iteration 3: THOUGHT recognizes all needed information is available →*

ACTION calls `final_answer(...)` → Loop terminates. This is ReAct in action: perceiving (observations), reasoning (thoughts), and acting (tool calls) in a goal-directed loop. The complete message history printed during execution is the agent's 'thought process' — fully transparent and debuggable.

Notebook 3: Prompt Chaining with Gate Checks

This notebook implements a complete prompt chaining workflow for automatically generating a Python data analysis script. It includes gate checks between each step and a feedback-driven refinement loop if validation fails.

In [1]: — Cell 1

```
import ast

# — Step 1 Prompt: Generate an Outline —————
OUTLINE_SYSTEM = 'You are an expert Python programming assistant.'

OUTLINE_USER = """I need a Python script that:
1. Reads data from a CSV file called 'sales_data.csv'
2. Calculates the average of a column called 'revenue'
3. Writes the result to a new CSV file called 'results.csv'

Please provide a numbered step-by-step outline for this script.
Be concise — one line per step. Include at least 4 steps."""

outline = call_llm(OUTLINE_SYSTEM, OUTLINE_USER)
print('=== GENERATED OUTLINE ===')
print(outline)
```

 *Explanation: This first cell is Step 1 of our prompt chain: generating a high-level outline. We keep the system prompt simple here since the task is a planning step, not a coding step. The user prompt is precise about what the script must do and includes a constraint ('at least 4 steps') to ensure we get a useful, detailed outline rather than a vague two-line response. In agentic workflows, having the LLM first plan before it executes is a form of Chain-of-Thought applied at the workflow level — the outline is the 'thinking step' before the 'coding step.' The output of this call becomes the input to the next step.*

In [2]: — Cell 2

```
# — Gate Check 1: Validate the Outline —————
def gate_check_outline(outline_text: str) -> tuple[bool, str]:
    """
    Validates that the outline is a numbered list with key required steps.
    Returns (passed: bool, feedback: str)
    """
    required_keywords = ['read', 'csv', 'average', 'write']
    # Check for numbered list format
    has_numbers = bool(re.search(r'^\d+\.', outline_text, re.MULTILINE))
    # Check all required concepts are mentioned
    missing = [kw for kw in required_keywords if kw.lower() not in
outline_text.lower()]
```

```

    if not has_numbers:
        return False, 'Outline is not a numbered list.'
    if missing:
        return False, f'Outline is missing required concepts: {missing}'
    return True, 'Outline validation passed.'

outline_passed, outline_feedback = gate_check_outline(outline)
print(f'Gate Check 1: {outline_feedback}')

if not outline_passed:
    raise ValueError(f'Step 1 Gate Check Failed: {outline_feedback}')

```

 **Explanation:** This cell implements Gate Check 1 — a programmatic validation that the outline meets our minimum quality requirements before we use it to generate actual code. The `gate_check_outline` function checks two things: (1) Structural check — is it a numbered list? An outline that isn't numbered is a signal the model misunderstood the task. (2) Content check — does it mention all the key concepts? If the outline doesn't mention 'read' and 'csv', the generated code will likely miss those requirements. The function returns both a boolean (pass/fail) and a human-readable feedback string. This feedback string is what we would inject into the retry prompt if the check fails — giving the LLM specific guidance on what to fix. The `raise ValueError` line halts the entire chain if the gate fails, preventing a cascade of bad outputs downstream.

In [3]: — Cell 3

```

# — Step 2 Prompt: Generate Code from Outline —————
CODE_USER = f"""Based on the following outline, write the complete Python script.
Requirements:
- Use only standard library modules (csv, statistics)
- Add comments for each major step
- Handle the case where the CSV file doesn't exist

OUTLINE:
{outline}
"""

generated_code = call_llm(OUTLINE_SYSTEM, CODE_USER)
print('=== GENERATED CODE ===')
print(generated_code)

```

 **Explanation:** This cell executes Step 2 of the chain: using the validated outline as input to generate actual Python code. Notice the f-string interpolation `{outline}` — this is how prompt chaining works at the code level. The output from the previous LLM call is embedded directly into the next prompt. We add additional constraints here that weren't in the outline prompt: use only standard library modules (so the script doesn't require package installation), add comments, and handle file-not-found errors. These constraints are better placed in the code-generation prompt than the outline prompt, because they are implementation details rather than design decisions. The output will be Python code as a string — which we'll validate in the next gate check.

In [4]: — Cell 4

```

# — Gate Check 2: Validate Code Syntax —————
def gate_check_syntax(code_text: str) -> tuple[bool, str]:
    """
    Uses Python's ast module to check for syntax errors without executing the code.
    Returns (passed: bool, feedback: str)
    """

```

```
"""
# Extract code from markdown code blocks if present
code_match = re.search(r'```(?:python)?\n(?:.*?)```', code_text, re.DOTALL)
clean_code = code_match.group(1) if code_match else code_text

try:
    ast.parse(clean_code)
    return True, 'Syntax validation passed.'
except SyntaxError as e:
    return False, f'SyntaxError on line {e.lineno}: {e.msg}'

# Run Gate Check 2
MAX_RETRIES = 3
for attempt in range(MAX_RETRIES):
    syntax_passed, syntax_feedback = gate_check_syntax(generated_code)
    print(f'Gate Check 2 (Attempt {attempt+1}): {syntax_feedback}')

    if syntax_passed:
        break # Gate passed — continue to next step

# Gate failed — inject error feedback into a refinement prompt
if attempt < MAX_RETRIES - 1:
    REFINE_USER = f"""The following Python code has a syntax error:
{generated_code}
Error: {syntax_feedback}
Please fix the syntax error and return only the corrected code."""
    generated_code = call_llm(OUTLINE_SYSTEM, REFINE_USER)
    print('Retrying with refined code...')
else:
    raise ValueError('Max retries exceeded. Could not generate valid code.')
```

 *Explanation: This cell implements Gate Check 2 with a retry loop — the most sophisticated validation in this notebook. Two important concepts come together here. First, `ast.parse()` performs static syntax analysis on the code string without actually executing it. This is safe (no risk of running malicious code), fast, and gives us precise error messages with line numbers. LLMs often wrap code in markdown code blocks (````python ... ````), so we use a regex to extract just the code before parsing. Second, the retry loop creates a feedback loop: if the syntax check fails, we don't just stop — we construct a new prompt that includes both the broken code AND the specific error message, then ask the LLM to fix it. The error message is the feedback that guides the correction. We cap at `MAX_RETRIES = 3` to prevent infinite loops. This combination of gate check + feedback-driven retry is the pattern you'll use repeatedly in production agentic systems.*

Notebook 4: Implementing LLM Feedback Loops

This notebook demonstrates a self-improving feedback loop where an LLM generates a professional email, a second LLM call evaluates it against quality criteria, and the feedback drives iterative refinement until the output meets the required standard.

In [1]: — Cell 1

```
# — Initial Draft Generation —————
DRAFT_SYSTEM = """You are a professional business communication specialist.
You write clear, concise, and appropriately formal business emails."""

DRAFT_USER = """Write a professional email to a potential client named Sarah Chen
requesting a 30-minute discovery call to discuss how our data analytics platform
could help her company. My name is Alex Kumar, I'm a Sales Engineer at DataFlow
Inc."""

draft_email = call_llm(DRAFT_SYSTEM, DRAFT_USER)
print('=== INITIAL DRAFT ===')
print(draft_email)
```

 *Explanation: This cell generates the first draft of our professional email. We give the LLM a clear role (professional business communication specialist) and a specific, contextual task with real names and a concrete request. The initial draft may be good — but it may also be too formal, too casual, too long, or missing important elements like a clear value proposition. Rather than manually revising it, we'll let the evaluator LLM in the next cell identify specific issues. Save the initial draft output to compare against later iterations.*

In [2]: — Cell 2

```
# — Evaluator LLM —————
EVAL_SYSTEM = """You are a strict business communication coach.
Evaluate emails and provide specific, actionable improvement feedback.
Always respond in this exact JSON format:
{
  'overall_score': <1-10>,
  'passes': <true if score >= 7, else false>,
  'issues': [<list of specific problems>],
  'suggestions': [<list of concrete improvements>]
}"""

def evaluate_email(email_text: str) -> dict:
    eval_user = f"""Evaluate this business email against these criteria:
    1. Professional but warm tone (not stiff or cold)
    2. Clear value proposition for the client
    3. Specific, reasonable call to action
    4. Appropriate length (under 200 words)
    5. Personalization (uses recipient's name)

    EMAIL:
    {email_text}
    """

    raw_response = call_llm(EVAL_SYSTEM, eval_user)
    # Clean and parse JSON response
    clean = re.sub(r"```json|```", "", raw_response).strip()
    # Replace single quotes with double quotes for valid JSON
    clean = clean.replace("'", '"')
    return json.loads(clean)
```

```
evaluation = evaluate_email(draft_email)
print(f"Score: {evaluation['overall_score']}/10")
print(f"Issues: {evaluation['issues']}")
print(f"Suggestions: {evaluation['suggestions']}")
```

 **Explanation:** This is the evaluator LLM — the second AI in our feedback loop. By instructing it to respond in a strict JSON format with specific fields (score, passes, issues, suggestions), we get machine-readable feedback that our code can act on. The passes field gives us a binary gate condition: is the email good enough to stop iterating, or do we need another revision? The issues list becomes the feedback content for the next refinement prompt. The re.sub call strips markdown code fences that the LLM sometimes wraps JSON in. Note the replace() call to convert single quotes to double quotes — LLMs sometimes output Python-style dicts with single quotes, which is not valid JSON. This kind of defensive parsing is essential in production systems where LLM output format is not 100% guaranteed.

In [3]: — Cell 3

```
# — Feedback Loop —
def run_email_feedback_loop(initial_draft: str, max_iterations: int = 4) -> str:
    current_draft = initial_draft
    iteration_log = []

    for i in range(max_iterations):
        print(f'\n--- Iteration {i+1} ---')
        evaluation = evaluate_email(current_draft)
        score = evaluation['overall_score']

        iteration_log.append({'iteration': i+1, 'score': score})
        print(f'Score: {score}/10 | Passes: {evaluation["passes"]}')

        # Exit if quality threshold met
        if evaluation['passes']:
            print('Quality threshold reached. Loop complete.')
            break

        # Build a refinement prompt incorporating specific feedback
        issues_text = '\n'.join(f'- {issue}' for issue in evaluation['issues'])
        suggestions_text = '\n'.join(f'- {s}' for s in evaluation['suggestions'])

        refine_user = f"""Please revise the following email.

CURRENT EMAIL:
{current_draft}

ISSUES TO FIX:
{issues_text}

SPECIFIC SUGGESTIONS:
{suggestions_text}

Return only the revised email, no commentary."""

        current_draft = call_llm(DRAFT_SYSTEM, refine_user)
```

```
print('Revised draft generated.')

print('\nScore progression:', [log['score'] for log in iteration_log])
return current_draft

final_email = run_email_feedback_loop(draft_email)
print('\n=== FINAL REFINED EMAIL ===')
print(final_email)
```

 **Explanation:** This cell contains the complete feedback loop orchestrator and is the culmination of the chapter's concepts. Walk through the logic carefully: (1) We evaluate the current draft and check if it scores ≥ 7 (our quality threshold). (2) If it passes, we exit the loop — quality has been achieved. (3) If it fails, we extract the specific issues and suggestions from the evaluation JSON and construct a detailed refinement prompt. Crucially, we include both the current draft AND the specific feedback — not just generic 'improve this email' instructions. The more specific the feedback, the better the revision. (4) The LLM produces a revised draft, which becomes `current_draft` for the next iteration. (5) The `iteration_log` tracks scores across iterations — we print this at the end as a 'score progression' list. You should see scores improving across iterations (e.g., [5, 7, 8]). This is the quantitative evidence that the feedback loop is working. If scores don't improve, examine the feedback being generated — it may be too vague or contradictory.

Chapter Summary

This chapter has taken you from the foundational question 'What is an AI Agent?' all the way through to implementing complete agentic workflows with feedback loops in Python. Here is a consolidated map of the concepts covered:

Concept	Key Takeaway
AI Agents	Autonomous systems with LLM reasoning + tools + memory + orchestration. They perceive, decide, and act in loops to achieve goals — far beyond passive chatbots.
Prompting	The primary interface for 'programming' LLM behavior. Every agentic behavior — role, reasoning style, tool use, output format — is controlled through prompts.
Role-Based Prompting	Assigning a specific persona to the LLM narrows its solution space and produces targeted, consistent, style-appropriate outputs. Use Role + Task + Format + Examples + Context.
Evaluation	Always build a ground truth dataset and evaluation pipeline. Use LLM-as-a-Judge for freeform outputs. Track traces. Iterate based on evidence.
Chain-of-Thought	Adding 'think step by step' dramatically reduces errors on multi-step reasoning tasks by making intermediate steps explicit and verifiable.

ReAct	Interleave Thought (reasoning) → Action (tool call) → Observation (result) loops to enable LLMs to use external tools and access real-world information.
Prompt Chaining	Decompose complex tasks into sequential, single-purpose prompts. Each output becomes the next input. Enables modularity, testing, and iteration.
Gate Checks	Programmatic validation between chain steps. Prevents error cascades. Types: Format, Content, Logic, Safety. Failed checks trigger retries with feedback.
Feedback Loops	Use evaluator LLMs or programmatic checks to generate feedback, then feed that feedback back as the prompt for a revised generation. Enables self-improvement without retraining.

Coming Up in Chapter 2

In the next chapter, we will build on everything covered here to design and implement complete, multi-agent systems.

You will learn how multiple specialized agents can collaborate — one agent researching, another writing, a third reviewing — in coordinated workflows that tackle tasks too complex for any single agent.

We will also introduce popular agentic frameworks (LangChain, LlamaIndex, AutoGen) and examine how they implement the patterns you have built from scratch in this chapter.